

SIMATS ENGINEERING ASSESSMENT TOOL 1

**COURSE NAME: DESIGN AND ANALYSIS OF ALGORITHMS
COURSE CODE: CSA0606**

COURSE OUTCOME COVERED

CO2: Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull

problems (BL1, BL2,BL3)

Assessment Tool Weightage: 40%

QUESTIONS: 10 Total Marks:100

Annexure A

ANALYTICAL PROBLEM SOLVING

Code of Conduct:

I B.Muni Chandini (192524283) (Name / Reg No) certify that this submission is my original work

and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: B.Muni Chandini

BINARY SEARCH

Unsuccessful Search in a Sorted List

Q46 | Design and Analysis of Algorithms

1. Problem Statement

Given a sorted dataset **[11, 22, 33, 44, 55, 66, 77]**, apply the Binary Search algorithm to search for the element **50**. Since the element does not exist in the list, demonstrate the complete process of reducing the search interval until the search is declared unsuccessful. Count the number of comparisons, analyse time complexity, verify correctness, and interpret the efficiency of Binary Search for unsuccessful cases.

2. Objectives

1. To understand the working principle of the Binary Search algorithm on a sorted array.
2. To demonstrate the step-by-step reduction of the search space when the target element is absent.
3. To count the exact number of key comparisons performed during the unsuccessful search.
4. To analyse the time complexity of Binary Search in the worst-case (unsuccessful) scenario.
5. To verify the correctness of the algorithm and interpret its efficiency compared to linear search.
6. To present a complete formal solution including mathematical model, pseudocode, flowchart, and test cases.

3. Inputs and Outputs

3.1 Inputs

- **arr[]** – A sorted array of integers: [11, 22, 33, 44, 55, 66, 77]
- **n** – Number of elements in the array ($n = 7$)
- **key** – The element to be searched ($\text{key} = 50$)

3.2 Outputs

- A Boolean / status message indicating whether the key was found or not.
- In this case: "Element 50 is NOT found" (Unsuccessful Search).
- Optionally: the number of comparisons performed (3).

4. Assumptions and Constraints

4.1 Assumptions

- The input array is already sorted in non-decreasing (ascending) order.
- All elements in the array are unique (no duplicates).
- Array indices start from 0.
- Integer division is used for computing the mid index (floor of average).
- The search key is an integer comparable with array elements.

4.2 Constraints

- $1 \leq n \leq 10^6$ (typical practical bound; here $n = 7$).
- Array elements and key are within standard integer range.
- The algorithm requires random-access memory (array indexing).

- Binary Search cannot be applied on an unsorted array without preprocessing.

5. Problem Analysis

The problem requires locating a specific key in a sorted collection. Because the data is ordered, we can exploit the ordering property to discard half of the remaining elements after each comparison. The key insight is that if the middle element is greater than the key, the key (if present) must lie in the left sub-array; if smaller, it must lie in the right sub-array. When the search interval becomes empty ($\text{low} > \text{high}$), we conclude the key is absent.

Key Observation: For the given array, 50 lies between 44 and 55. The algorithm will eventually examine both neighbouring values and then terminate, confirming absence.

6. Mathematical Model

Let $A[0 \dots n-1]$ be a sorted array and k the search key.

Define the search interval at any step as the closed index range $[L, H]$, where initially $L = 0$ and $H = n - 1$.

At each iteration:

- Compute $\text{mid} = \lfloor (L + H) / 2 \rfloor$
- If $A[\text{mid}] = k \rightarrow$ Success (return mid)
- If $A[\text{mid}] < k \rightarrow L \leftarrow \text{mid} + 1$ (discard left half including mid)
- If $A[\text{mid}] > k \rightarrow H \leftarrow \text{mid} - 1$ (discard right half including mid)

Termination conditions:

- Successful: $A[\text{mid}] = k$
- Unsuccessful: $L > H$ (empty interval)

The maximum number of iterations is bounded by $\lceil \log_2(n + 1) \rceil$, giving the recurrence:

$$T(n) = T(\lfloor n/2 \rfloor) + \Theta(1) \quad \text{with} \quad T(1) = \Theta(1)$$

By the Master Theorem ($a = 1$, $b = 2$, $f(n) = \Theta(1)$), the solution is $T(n) = \Theta(\log n)$.

7. Algorithm Selection

Three common searching techniques were considered:

Algorithm	Time Complexity	Requires Sorted?	Suitability
Linear Search	$O(n)$	No	Simple but slow
Binary Search	$O(\log n)$	Yes	Optimal choice
Jump / Interpolation	$O(\sqrt{n}) / O(\log \log n)$	Yes	Overkill for $n=7$

Selected Algorithm: Binary Search — because the array is sorted and we need the most efficient deterministic method with logarithmic comparisons.

8. Algorithm / Pseudocode

Algorithm BinarySearch($A[0 \dots n-1]$, key):

1. Set $\text{low} \leftarrow 0$, $\text{high} \leftarrow n - 1$
2. While $\text{low} \leq \text{high}$ do
3. $\text{mid} \leftarrow \lfloor (\text{low} + \text{high}) / 2 \rfloor$
4. if $A[\text{mid}] = \text{key}$ then

```

5.     return mid          // Successful
6.     else if A[mid] < key then
7.         low ← mid + 1
8.     else
9.         high ← mid - 1
10. end while
11. return -1              // Unsuccessful

```

Alternative recursive formulation:

BinarySearch(A, low, high, key):

```

    if low > high then return -1
    mid ← [(low + high) / 2]
    if A[mid] = key then return mid
    if A[mid] < key then return BinarySearch(A, mid+1, high, key)
    else return BinarySearch(A, low, mid-1, key)

```

9. Flowchart (Textual Representation)

The control flow of the iterative Binary Search can be visualised as follows:



10. Step-by-Step Execution

Array: [11, 22, 33, 44, 55, 66, 77] Indices: 0 1 2 3 4 5 6 Key = 50

Step 1 – Initialization & First Comparison

low = 0, high = 6

mid = [(0 + 6) / 2] = 3 → A[3] = 44

Comparison 1: 50 > 44 ⇒ discard left half (including 44)

Update: low ← 4

Step 2 – Second Comparison

low = 4, high = 6

mid = [(4 + 6) / 2] = 5 → A[5] = 66

Comparison 2: $50 < 66 \Rightarrow$ discard right half (including 66)

Update: $\text{high} \leftarrow 4$

Step 3 – Third Comparison

$\text{low} = 4, \text{high} = 4$

$\text{mid} = \lfloor (4 + 4) / 2 \rfloor = 4 \rightarrow A[4] = 55$

Comparison 3: $50 < 55 \Rightarrow$ discard right half (including 55)

Update: $\text{high} \leftarrow 3$

Step 4 – Termination

Now $\text{low} = 4$ and $\text{high} = 3$

Condition $\text{low} > \text{high}$ is TRUE $\Rightarrow$ search interval is empty

Result: Element 50 is NOT present in the array (Unsuccessful Search).

Summary Table of Execution

Step	low	high	mid	A[mid]	Decision
1	0	6	3	44	$50 > 44 \rightarrow \text{low} = 4$
2	4	6	5	66	$50 < 66 \rightarrow \text{high} = 4$
3	4	4	4	55	$50 < 55 \rightarrow \text{high} = 3$
4	4	3	—	—	$\text{low} > \text{high} \rightarrow \text{STOP (Not Found)}$

11. Complexity Analysis

11.1 Time Complexity

In every iteration the search space is halved. After at most $\lceil \log_2(n) \rceil + 1$ comparisons the interval becomes empty or the key is found.

- **Best Case:** $O(1)$ – key is at the middle position on the first comparison.
- **Average Case:** $O(\log n)$
- **Worst Case (Successful or Unsuccessful):** $O(\log n)$

For $n = 7$: $\lceil \log_2 7 \rceil + 1 = 2 + 1 = 3$ comparisons (exactly what we observed).

11.2 Space Complexity

- **Iterative version:** $O(1)$ auxiliary space (only a few index variables).
- **Recursive version:** $O(\log n)$ stack space due to recursive calls.

11.3 Comparison with Linear Search

Metric	Linear Search	Binary Search
Worst-case comparisons ($n=7$)	7	3
Time Complexity	$O(n)$	$O(\log n)$
Requires Sorted Data	No	Yes

12. Correctness Analysis

12.1 Loop Invariant

At the beginning of every iteration the following invariant holds:

If the key exists in the original array, then it must lie in the sub-array $A[\text{low} \dots \text{high}]$.

Initialization: low = 0, high = n-1 covers the whole array → invariant true.

Maintenance: After each comparison we discard only the half that cannot contain the key, so the invariant is preserved.

Termination: When low > high the interval is empty. By the invariant the key cannot be present → algorithm correctly reports “not found”.

12.2 Verification for the Given Instance

Manual inspection of [11, 22, 33, 44, 55, 66, 77] confirms that 50 is not present. The algorithm examined the two nearest neighbours (44 and 55) and then terminated, which is the expected behaviour. All three comparisons produced consistent half-elimination decisions. Hence the result is verified correct.

13. Test Cases

TC	Array	Key	Expected Output	Remarks
1	[11,22,33,44,55,66,77]	50	Not Found	Primary case (unsuccessful)
2	[11,22,33,44,55,66,77]	44	Found at index 3	Successful – middle element
3	[11,22,33,44,55,66,77]	11	Found at index 0	Successful – first element
4	[11,22,33,44,55,66,77]	77	Found at index 6	Successful – last element
5	[11,22,33,44,55,66,77]	10	Not Found	Key smaller than all
6	[11,22,33,44,55,66,77]	100	Not Found	Key larger than all
7	[11]	11	Found at index 0	Single-element success
8	[11]	5	Not Found	Single-element failure

14. Results

For the primary problem instance:

- **Search Key:** 50
- **Status:** **Unsuccessful (Element Not Found)**
- **Number of Comparisons:** 3
- **Elements Examined:** 44, 66, 55
- **Final Interval:** low = 4, high = 3 (empty)

The algorithm correctly reported that 50 does not exist in the sorted list. All additional test cases also produced the expected outputs, confirming the reliability of the implementation.

15. Conclusion

Binary Search is an efficient divide-and-conquer algorithm that exploits the sorted nature of the data to locate (or determine the absence of) a key in logarithmic time. In the given problem, the search for 50 in the array [11, 22, 33, 44, 55, 66, 77] required only three comparisons before the search interval became empty, correctly concluding that the element is not present.

Even in the unsuccessful case, Binary Search remains far superior to Linear Search (3 vs. up to 7 comparisons). The algorithm’s correctness is guaranteed by the loop invariant that the key, if present, always lies within the current interval [low, high]. When that interval shrinks to empty, absence is proven.

This case study demonstrates that Binary Search is the method of choice whenever the input is sorted and fast searching is required, whether the outcome is successful or unsuccessful.